

Final Project Writeup

Find Your Next...

Amelia, Maia, Kendall, Najmo

- [Github Link](#)
- [Azure Web App Link](#)

Application Overview

“Find Your Next...” is an entertainment recommendation engine designed to bridge the gap between books and movies. Traditional recommendation systems like Goodreads or Netflix’s “What to watch” operate within a single media type, but this application allows users to pivot between both. In our application, you can still do book to book or movie to movie like traditional systems. However, you can also find books based on movies that you liked or movies based on books that you liked.

Architecture

Front End:

- The front ended was built using HTML templates powered by Jinja2 and styled with CSS
 - Jinja allows us to inject data directly from our Python backend into the HTML, making the pages dynamic rather than static
 - The pages were styled with a shared CSS to make sure that all of the pages were visually consistent across the site and user friendly
- **Home Page:**
 - The home page allows users to select whether they were entering a book or a movie and what type of recommendation they wanted in return
 - We thought that a dropdown menu would be the best to allow the user to pick between a book or a movie
 - A text field was used for the search bar for the user to write their book or movie title
 - A genre dropdown allows the user to make their recommendation more specific when they entered a book and wanted a movie in return

- We use Jinja to pre-load and render the “Featured Movies” section dynamically when the app starts
- **Results Page:**
 - The results page displays recommendations based entirely on the api (for our first test we implemented fake movie posters just to have a better visual)
 - The results are displayed in a grid of cards, created by a Jinja loop
 - Each result includes:
 - A poster
 - The title of the book or movie
 - The release year + rating
 - The CSS handles the layout
 - A message will be displayed if there are no recommendations found. (this shouldn't happen but just in case)
- **Details Page:**
 - Selecting a result opens a details page for that book or movie
 - The information you could find on this page:
 - Title
 - Description
 - Poster
 - A link to a better description on google
 - There are buttons at the top of the page that allow you to go back to the previous page

Backend:

- **Flask Server (app.py):** Handles routing and HTTP requests. Processes inputs from frontend to determine which function to follow, and renders the appropriate template with the function output.
- **Custom Functions (functions.py):** contains core logic and API integration functions that are called by the user.

Data & API Integration

External API Sources:

- **The Movie Database (TMDb) API:** used for movie metadata, posters, and “movie-to-movie” recommendations.
- **Big Book API:** used for book metadata, “book-to-book” recommendations, and genre-based book searches.

- **OpenLibrary Covers API:** used only to fetch book cover images using ISBNs retrieved from the Big Book API

SQL Server and Database: The two primary APIs could not communicate with each other on their own. TMDb uses inter-based IDs for genres (ex: 878 for Science Fiction), while Big Book API uses string-based labels. There is also a discrepancy between the names of the genres (ex: comedy for movies = humor for books). To solve this, we implemented a **GenreMap table** using an Azure SQL Database. The columns are:

- BookGenreName (VARCHAR, Primary Key): The string identifier used by the Book API
- TMDbGenreID (INT): The integer identifier used by the Movie API that *we decided* corresponds with the primary key. Multiple TMDb genres can correspond with the same book genre.

Logic Pipelines

1. **Movie → Movie:** direct API call, TMDb has a native recommendation algorithm
2. **Book → Book:** direct API call using Big Book API's find_similar_books endpoint. Fetch the results' ISBNs to query OpenLibrary API for the covers.
3. **Movie → Book:** take title as input, find genre ID of that movie, SQL lookup, search book API using the corresponding genre from table, fetch the results' ISBNs to query OpenLibrary API for the covers, return desired number of results in order of rating.
4. **Book → Movie:** The Big Book API does not have the functionality to fetch the genre from a book title. Since we were doing recommendations based on genre, we had to resort to taking genre as an input from the user. So we take title AND genre as input, find genre name using SQL lookup, search movie API using the corresponding genreID from table, return desired number of results in order of rating.
5. **“Featured” Movies:** we wanted to fill space on our home screen, so we took advantage of the built in “trending” function in the TMDb API to display the top 4 currently trending movies (these change daily).

User Acceptance Testing and Feedback

We only received feedback from two users: one for just the front end, and one for the complete application. The front end feedback recommended that we include tags with information about each book or movie before you click on them to pull up the description. They said that it would make navigation easier for users. We ended up

including the rating and year as a tag for featured movies, and year as a tag for books and movies. The reason for limiting the information in the tags is because it requires additional API calls, and we are very limited on API calls in the free tier. This same API call usage limits our response to the second user's feedback. They said that they would like to increase the max amount of results displayed to more than 12, because there is a possibility that you have already seen/read the majority of the results. This is a good point, but we are again limited by our API calls. We get 50 calls per day for the BigBook API, so allowing anything more than 12 would burn through the credits very quickly.

Automated Testing

To ensure the reliability of the backend, we implemented automated testing using pytest. The primary goal of these tests was to verify that each Flask endpoint behaved correctly under different input conditions without relying on external APIs or the Azure SQL database. Because the app communicates with multiple third-party services, we used mocking to simulate responses from TMDb, BigBookAPI, and database queries. This allowed the tests to run quickly and deterministically while still validating that our routes handled data correctly, rendered the appropriate templates, and returned the expected status codes. Each major endpoint (homepage, recommend, detail) has at least one automated test, as required. These tests cover successful recommendation flows, empty or missing inputs, invalid IDs, and error-handling paths such as the 404 case when an item is not found. We chose not to write automated tests for the helper functions in `functions.py` because they rely heavily on external APIs and network responses; instead, we test their integration through the Flask routes.

Manual Testing

Manual testing was performed to ensure that the application functions correctly and is easy to use. The testing focused on verifying key features such as searching for books and movies, generating recommendations, navigating between pages, cross media recommendations, and the featured movies on the homepage. Different input scenarios were tested including valid entries, empty searches, and empty fields to confirm that the application ran all the cases correctly. We also checked that the search results and recommendations loaded correctly and that cross media recommendations displayed relevant items. Additional testing included verifying that the interface on the screen actually worked and did what they were supposed to do. The goal of manual testing was to identify any errors that the user might encounter and make the experience that our users had a friendly experience.

Final Project App Testing Plan

Group Name: Find Your Next

App Link:

<https://eibert-amelia-final-project-webapp-evhad4ezfkfbwd9.westus3-01.azurewebsites.net/>

Test Cases

	Test Case Instructions	Expected Result
Test Case 1: Search Movies	1. Open the app link 2. In the “What are you looking for?” Enter Movie —> Movie 3. There you should enter the movie title	1. Movie recommendations should appear and you can choose between 3, 4, 9, 12 2. When you click on a title it should have a ratings, summary, and a “search on the web” link
Test Case 2: Search Books	1. Open the app link 2. In the “What are you looking for?” Enter Movie —> Movie 3. There you should enter the book title	1. Book recommendations should appear and you can choose between 3, 4, 9, 12 2. When you click on a title it should have a ratings, summary, and a “search on the web” link
Test Case 3: Cross Media Recommendation	1. Open the app link 2. In the “What are you looking for?” Enter Movie —> Book Or Book —> Movie 3. There you should enter the movie or book title	1. Recommendations should appear for whatever input you put in and you can choose between 3, 4, 9, 12 2. When you click on a title it should have a ratings, summary, and a “search on the web” link

For the final test case, perform general testing of the application with the intention to break things.

Test Results

	Pass/Fail	Notes
Test Case 1	Pass	
Test Case 2	Pass	
Test Case 3	Pass	

General Testing Results

Issues/Bugs	App Feedback
	- Tags for each movie or book before you click on them - Show more than 12 recommendations

Responsibilities

Each team member contributed to a different core area of the project. Maia focused on the front-end interface, designing and implementing the HTML templates, layout, and overall user experience across the different pages. Kendall developed much of the backend logic, including the recommendation functions and processing pipelines that power the app's core functionality. Amelia handled the data and API integration work, connecting the application to TMDB, BigBookAPI, and the Azure SQL database, as well as creating the genre-mapping table used for cross-media recommendations. She also developed the automated backend tests to ensure the reliability and correctness of the application's routes and features. Najmo worked on manual testing of the application.